

13. Design of counter with increment of N using Vivado HLS

Aim

To design and implement an 8-bit counter that increments by a programmable value (N) using Xilinx Vivado HLS, convert the C++ design into RTL, and verify its functionality using a Verilog testbench and waveform simulation.

Software Required

- Xilinx Vivado Design Suite
- Vivado HLS
- Vivado Simulator (XSim)
- Verilog HDL

Procedure

1. C++ Design Using Vivado HLS
 - A C++ program was written using the `ap_uint<8>` data type to implement an 8-bit counter.
 - The counter increments by a user-defined value N on each activation.
 - A static variable was used to store the counter value, ensuring it is retained across clock cycles.
2. High-Level Synthesis
 - The C++ code was synthesized using Vivado HLS.
 - A control interface was enabled to generate handshake signals for hardware operation.
 - The design was pipelined to improve performance.
3. RTL Generation
 - Vivado HLS converted the C++ counter design into Verilog RTL.
 - The generated RTL implemented an 8-bit registered counter with wrap-around behavior.
4. Verilog Testbench Development
 - A Verilog testbench was written to provide clock and control signals.
 - Different values of increment N were applied to the counter.
5. Simulation and Waveform Verification
 - The design was simulated using Vivado Simulator.
 - Counter output waveforms were observed and verified for correct increment and overflow behavior.

C++ program for an upcounter with an increment of N

```
#include <ap_int.h>

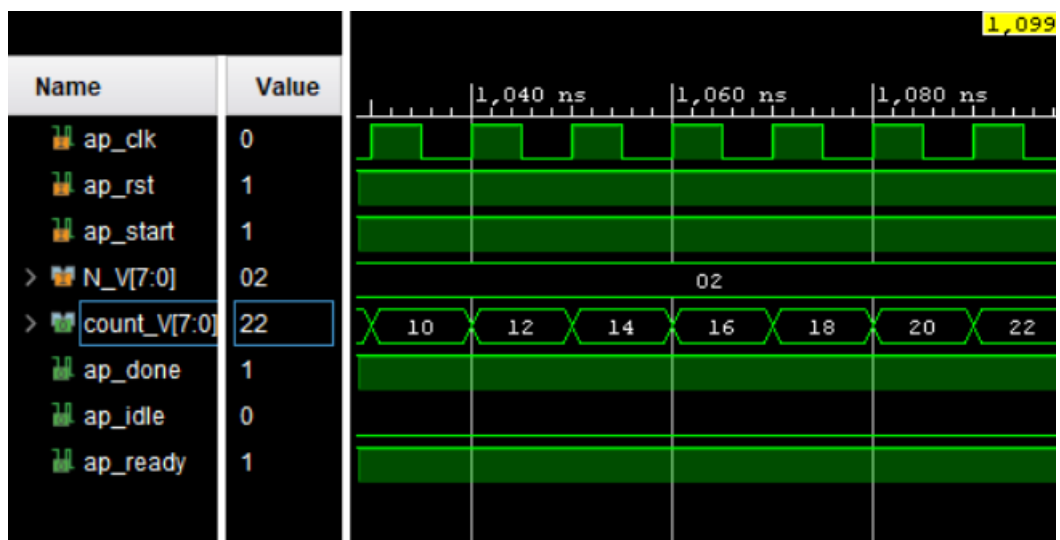
void counter_inc_by_n(
    ap_uint<8> N,
    ap_uint<8> &count
){
    #pragma HLS INTERFACE ap_ctrl_hs port=return
    #pragma HLS INTERFACE ap_none port=N
    #pragma HLS INTERFACE ap_none port=count
    #pragma HLS PIPELINE

    static ap_uint<8> cnt = 0;

    cnt = cnt + N; // wraps automatically at 8 bits

    count = cnt;
}
```

Testbench output



Result

The 8-bit counter incremented by a programmable value was successfully designed and verified. Simulation waveforms confirmed that the counter increased correctly by N on each clock cycle and wrapped around after reaching the maximum 8-bit value. The results validated the correct functionality of the HLS-generated RTL design.